

agent: discovery-technical-debt-agent cli: Claude Code CLI llm: claude-opus-4-6
run_id: 20260818T185214_j4bz7e generated_at: 2026-08-18T13:28:23.553Z

8. Technical Debt Analysis

Objective: Establish the prerequisites for an Agentic Harness & Marketplace initiative — code repository health, third-party tool usage, AI tool usage, database usage, and development environment readiness.

Date: 2026-08-18 18:58:33 IST | **Scope:** shende-shweta/FSDKC — PHP 8.3 / Laravel 12 backend, React 19.2 / TypeScript / Vite frontend, Node.js 22 Express dev-API, MariaDB 11, MongoDB 7, Docker Compose, GitHub Actions CI

Executive Summary

EXECUTIVE SUMMARY

The Klearcom monolithic platform has a sound foundational architecture — Docker Compose orchestration, a GitHub Actions CI pipeline, committed lock files for Node.js packages, and `.env.example` files for both the backend and dev-API. However, several structural gaps collectively lower its agentic-harness readiness. The backend `composer.lock` is not committed, eliminating reproducible PHP dependency installs. No linter, formatter, or pre-commit hook is configured or enforced anywhere in the stack, so code style drifts immediately. Branch-protection artifacts (CODEOWNERS, PR templates, required-checks config) are entirely absent. PHPStan is declared as a dev-dependency but is never wired into CI or any local command. The database uses a single monolithic `init.sql` instead of versioned migrations, and the MariaDB schema has no secondary indexes beyond primary keys. The codebase does ship AI-assistance guides (`AGENTS.md` at root and per-module), positioning it ahead of most repositories for agentic readiness, but the lack of enforced style and a missing `composer.lock` mean agent-authored changes cannot be deterministically verified. The overall verdict is **Moderate** — no single dimension is catastrophically broken, but the cumulative gaps across repository health, development environment, and database hygiene block safe, automated agent workflows today.

Yes

TOP-LEVEL .GITIGNORE
PRESENT

1

CI/CD WORKFLOWS FOUND

9 / 7

THIRD-PARTY PACKAGES
DECLARED / WIRED

Partial

.ENV.EXAMPLE PRESENT

OVERALL CODEBASE RATING — TECHNICAL DEBT & AGENTIC READINESS

Moderate

All five dimensions rated Moderate — the breadth of 1–2 gaps per dimension (missing composer.lock, no style enforcement, no migration framework, partial .env.example coverage) prevents any dimension from reaching Good, but none crosses the High Risk threshold individually.

Readiness Benchmark Ratings

#	Dimension	GOOD	MODERATE	HIGH RISK	Measured	Rating
D1	Code Repository Health	all checks pass	1–2 gaps	3+ gaps / no CI	CI present and runs tests + build; <code>.gitignore</code> covers deps and env; Node lock files committed. Gaps: <code>composer.lock</code> missing; no CODEOWNERS / PR template. (2 gaps)	MODERATE
D2	Third-Party Tool Usage	mostly wired & current	some unused/unwired	many unused/unmaintained	7 of 9 declared packages are actively wired in application code. PHPStan and <code>dotenv</code> (dev-api) are declared but not wired into any enforced workflow.	MODERATE
D3	AI Tool / Agentic Readiness	ready	partial	not ready	Root and per-module <code>AGENTS.md</code> files exist. <code>.kiro/</code> directory present (mostly empty). Two product modules (Discovery, Connect) form structurally uniform, enumerable units. Missing: enforced style and deterministic PHP builds block safe agent output verification.	MODERATE
D4	Database Usage	sound	some gaps	no constraints / shared flat schema	Foreign keys present on child tables. Schema uses ENUM constraints and NOT NULL. Gaps: no secondary indexes on MariaDB tables; single monolithic <code>init.sql</code> instead of versioned migrations; MongoDB collections lack schema validation.	MODERATE
D5	Development Environment	reproducible	partial	manual / fragile	Docker Compose present with health checks. <code>backend/.env.example</code> and <code>dev-api/.env.example</code> present. Gaps: no root or frontend <code>.env.example</code> ; no linter/formatter/pre-commit hook configured anywhere; <code>composer.lock</code> not committed.	MODERATE

No additional readiness gaps beyond the standard dimensions were observed.

8.1 Code Repository

Check	Finding	Evidence
<code>.gitignore</code> coverage	Present and adequate. Covers <code>backend/vendor/</code> , <code>backend/.env</code> , <code>node_modules/</code> , <code>frontend/dist/</code> , <code>dev-api/.env</code> , <code>*.log</code> , and root <code>.env</code> — 8 entries spanning all three sub-projects. Prevents committing dependency dirs, build output, and secret files.	<code>.gitignore:1-8</code>
CI/CD presence	Present. Single workflow <code>.github/workflows/ci.yml</code> triggers on push to <code>main</code> / <code>master</code> and all PRs. Backend job: sets up PHP 8.3, installs MongoDB extension via PECL, runs <code>composer install</code> , copies <code>.env.example</code> , generates app key, and executes <code>vendor/bin/phpunit</code> against a MariaDB 11 service container. Frontend job: sets up Node 22, runs <code>npm ci</code> (with <code>npm install</code> fallback), and runs <code>npm run build</code> which executes <code>tsc --noEmit && vite build</code> . No lint step, no PHPStan step, and no dev-api tests are included.	<code>.github/workflows/ci.yml:1-42</code>
Branch protection signals	Absent. No <code>CODEOWNERS</code> file, no <code>.github/PULL_REQUEST_TEMPLATE.md</code> , no <code>.github/branch-protection*</code> or <code>ruleset</code> config visible in the repository. The <code>.github/</code> directory contains only the <code>workflows/</code> subdirectory. This means unreviewed code can land on <code>main</code> without a required review or passing status check.	<code>.github/</code> directory contains only <code>workflows/ci.yml</code>
Lock files committed	Partial. Three Node.js lock files committed: <code>package-lock.json</code> at root (11 KB), <code>frontend/package-lock.json</code> (65 KB), and <code>dev-api/package-lock.json</code> (54 KB). <code>composer.lock</code> is not committed for the Laravel backend — <code>.gitignore</code> does not exclude it, so it was simply never generated or committed. This means <code>composer install</code> in CI resolves dependencies at whatever version is latest, producing non-reproducible builds that may differ from the developer's local environment.	<code>ls</code> confirms <code>backend/composer.lock</code> absent; three <code>package-lock.json</code> files present

8.2 Third-Party Tools Usage

Backend (PHP / Laravel 12) — `backend/composer.json`

Package	Declared	Actually Wired?	Debt
<code>laravel/framework</code> ^12.0	Y	Y — routing, Eloquent ORM, middleware, CORS, queue dispatch, config used throughout <code>backend/app/</code> (all controllers, models, service providers)	None
<code>mongodb/mongodb</code> ^2.0	Y	Y — <code>app/Services/MongoService.php:15</code> instantiates <code>MongoDB\Client</code> , reads/writes to <code>transcripts</code> , <code>test_events</code> , <code>call_diagnostics</code> collections; used by <code>MongoController</code> , <code>RealTimeTestService</code> , <code>DiscoveryController</code> , <code>ConnectController</code>	None

<code>phpunit/phpunit</code> ^11.0 (dev)	Y	Y — <code>phpunit.xml</code> defines Unit test suite; CI runs <code>vendor/bin/phpunit</code> ; 2 test files exist in <code>backend/tests/Unit/</code>	Tests are trivial/non-deterministic but the tool is wired
<code>phpstan/phpstan</code> ^2.0 (dev)	Y	N — no <code>phpstan.neon</code> or <code>phpstan.neon.dist</code> config file exists anywhere in the repo; CI does not run <code>phpstan analyse</code> ; no script in <code>composer.json</code> references it	Declared but completely unused — provides no static-analysis value until configured and enforced

Frontend (React / TypeScript) — `frontend/package.json`

Package	Declared	Actually Wired?	Debt
<code>react</code> ^19.2.3 + <code>react-dom</code> ^19.2.3	Y	Y — <code>main.tsx</code> renders root; all pages and components import React	None
<code>@tanstack/react-query</code> ^5.62.0	Y	Y — <code>QueryClientProvider</code> in <code>main.tsx</code> ; <code>useQuery</code> / <code>useMutation</code> in <code>DashboardPage</code> , <code>DiscoveryPage</code> , <code>ConnectPage</code> , <code>MongoStatus</code>	None
<code>react-router-dom</code> ^7.1.0	Y	Y — <code>BrowserRouter</code> in <code>main.tsx</code> ; <code>Routes</code> / <code>Route</code> / <code>NavLink</code> in <code>App.tsx</code>	None
<code>zustand</code> ^5.0.2	Y	Y — <code>src/store/uiStore.ts</code> provides <code>selectedDiscoveryId</code> / <code>selectedMonitorId</code> state	None
<code>typescript</code> ^5.7.2 + <code>vite</code> ^6.0.3 + <code>@vitejs/plugin-react</code> ^4.3.4 (dev)	Y	Y — <code>vite.config.ts</code> configures plugin; CI runs <code>tsc --noEmit && vite build</code>	None

Dev-API (Node.js / Express) — `dev-api/package.json`

Package	Declared	Actually Wired?	Debt
<code>express</code> ^4.21.2	Y	Y — <code>src/server.js</code> creates Express app, mounts all routes	None
<code>cors</code> ^2.8.5	Y	Y — <code>app.use(cors())</code> at <code>src/server.js:17</code>	Wired but with default <code>*</code> origin — allows any origin to call the API; a security concern but not a wiring gap
<code>mongodb</code> ^6.12.0	Y	Y — <code>src/mongo.js</code> uses <code>MongoClient</code> for all document operations across 3 collections	None
<code>mongodb-memory-server</code> ^10.1.4	Y	Y — <code>src/mongo.js:31-36</code> uses <code>MongoMemoryServer.create()</code> as Atlas fallback when <code>MONGODB_URI</code> is absent	None
<code>dotenv</code> ^16.4.7	Y	N — declared in <code>package.json</code> but never <code>import</code> ed in application code; <code>src/seed.js:1</code> imports <code>dotenv/config</code> but	Unnecessary dependency — <code>seed.js</code> imports it but <code>server.js</code> (the main entry

	the npm scripts use Node 22's native <code>--env-file=.env</code> flag instead, making the runtime dependency redundant	point) does not; the <code>--env-file</code> flag makes it redundant
--	---	--

Summary: 9 infrastructure/security-relevant packages declared across the three workspaces; 7 are actively wired. PHPStan (backend) and dotenv (dev-api) are declared but not functionally wired into any enforced workflow.

8.3 AI Tool Usage & Agentic Readiness

Existing AI Tooling

Artifact	Path	Maturity
Root agent guide	<code>AGENTS.md</code>	Present — documents platform purpose, stack, module locations, and 3 coding conventions (no <code>extract()</code> , functional React only, manual SQL schema)
Backend module guides	<code>backend/app/Modules/Discovery/AGENTS.md</code> , <code>backend/app/Modules/Connect/AGENTS.md</code>	Present — list API endpoints, data stores, KPIs, and conventions per module
Frontend module guides	<code>frontend/src/modules/Discovery/AGENTS.md</code> , <code>frontend/src/modules/Connect/AGENTS.md</code>	Present — list screens, state management, and API integration per module
<code>.kiro/</code> directory	<code>.kiro/settings/mcp-bundles/.gitignore</code>	Skeleton only — MCP bundles directory exists but contains only a <code>.gitignore</code> ; no active Kiro configuration
Codebase audit	<code>docs/CODEBASE_AUDIT_ISSUES.md</code>	Thorough 14-point audit covering backend architecture, frontend issues, security gaps, and testing — provides a ready-made work queue

No other AI tooling (`.cursor/` , `.github/copilot*` , `CLAUDE.md` , codegen scripts) is present.

Convention Violations Found

2 files affected.

File	Line(s)	Issue	Action
<code>backend/app/Http/Controllers/Api/LegacyReportController.php</code>	22	<code>extract()</code> usage violates AGENTS.md convention — "No <code>extract()</code> in PHP"	Replace <code>extract()</code> with explicit array destructuring
<code>backend/app/Legacy/LegacyDataMapper.php</code>	6, 12, 24	<code>extract()</code> usage violates AGENTS.md convention — "No <code>extract()</code> in PHP"	Replace <code>extract()</code> with explicit array destructuring

The `AGENTS.md` explicitly states "No `extract()` in PHP — use explicit destructuring" but `extract()` appears in 2 production files: `backend/app/Legacy/LegacyDataMapper.php:12,22` (both `extract($row, EXTR_SKIP)` and unguarded `extract($context)`) and `backend/app/Http/Controllers/Api/LegacyReportController.php:21` (`extract($filters)` on raw `$request->all()` — injects all request parameters as local variables, a security risk).

Enumerable, Isolated Units of Work

The codebase has a clear modular structure: two product modules (Discovery, Connect) with mirrored backend controllers + models + frontend pages. This creates structurally uniform units that an agent could target systematically:

- **Controller → Service extraction** — 6 controllers follow the same pattern, each with business logic that should move to a service layer
- **Duplicate code consolidation** — `buildTree()` exists in 3 locations (`DiscoveryController.php:87-101` , `LegacyReportController.php:77-91` , `dev-api/src/store.js:54-64`); reachability formula is duplicated 3× (`ConnectController.php:65-74` , `RealTimeTestService.php:117-125` , `dev-api/src/realtime.js:143-151`)
- **Per-module API hook generation** — 2 modules × (frontend + backend) with clear API boundaries
- **Test generation per service** — each service has well-defined input/output boundaries suitable for automated test generation

Blockers for Safe Agent Operation

1. **No enforced code style** — an agent cannot verify its output matches project conventions because no linter or formatter is configured. An agent-authored PR could introduce style drift with no automated way to catch it.
2. **Non-reproducible PHP builds** — without `composer.lock` , CI may resolve different dependency versions than the agent's local environment, causing phantom failures.
3. **Weak test gate** — CI runs `phpunit` but the existing tests are trivial (hardcoded assertions in `HealthTest.php`) or non-deterministic (`random_int()` in `ReachabilityCalculationTest.php`), so a passing CI run provides low confidence that agent changes are correct.

Verdict

The codebase is **partially ready** for agentic workflows. The modular structure and AGENTS.md documentation are above average. The missing enforced style, absent `composer.lock` , and weak test suite must be addressed before an agent harness can safely accept machine-authored changes.

8.4 Database Usage

MariaDB (Relational — `docker/mariadb/init.sql`)

Check	Finding	Evidence
Schema design — foreign keys	Present on child tables. <code>discovery_nodes.discovery_job_id</code> → <code>discovery_jobs(id) ON DELETE CASCADE</code> (<code>init.sql:29-30</code>); <code>connect_check_results.connect_monitor_id</code> → <code>connect_monitors(id) ON DELETE CASCADE</code> (<code>init.sql:46-47</code>). However, <code>discovery_nodes.parent_id</code> is a self-referencing nullable column with no explicit foreign key constraint — orphan nodes can be created without any database-level guard.	<code>docker/mariadb/init.sql:22-30</code> , <code>docker/mariadb/init.sql:40-47</code>
Schema design —	Only primary keys. No secondary indexes on <code>discovery_jobs.status</code> , <code>discovery_nodes.discovery_job_id</code> , <code>connect_monitors.status</code> ,	<code>docker/mariadb/init.sql</code> — no <code>CREATE INDEX</code> statements;

indexes	<code>connect_monitors.country_code</code> , <code>connect_check_results.connect_monitor_id</code> , or <code>connect_check_results.checked_at</code> . The application queries filter and sort by these columns (e.g. <code>ConnectCheckResult::where('connect_monitor_id', \$id)->orderByDesc('checked_at')</code>) in <code>ConnectController.php:58-61</code> , so full table scans are expected as data grows.	controllers filter on un-indexed columns
Migration hygiene	No migration system. Schema is defined as a single <code>docker/mariadb/init.sql</code> loaded via Docker endpoint <code>docker-entrypoint-initdb.d/</code> . No <code>database/migrations/</code> directory exists in the backend. The <code>README.md</code> and <code>AGENTS.md</code> both codify this as intentional: "Schema via manual SQL init — matching Klearcom's manual migration practice." Changes require editing the monolithic SQL file — no rollback path, no change history, no guard against destructive alterations.	<code>docker-compose.yml:26</code> , <code>README.md:17</code> , <code>AGENTS.md:24</code>
Data ownership	Single flat schema. All 5 MariaDB tables share one database (<code>klearcom</code>) with no namespace, schema, or prefix separation between Discovery and Connect domains. While tables are logically scoped by naming convention (<code>discovery_*</code> vs <code>connect_*</code>), there is no schema-level isolation that would support safe extraction into separate services later. The <code>users</code> table has no FK relationship to any module table.	<code>docker/mariadb/init.sql</code> — all <code>CREATE TABLE</code> in default schema
Seed data hygiene	Seed data baked into <code>init.sql</code> alongside schema DDL (<code>init.sql:49-78</code>). The <code>INSERT</code> statements are idempotent only because Docker endpoint runs <code>init.sql</code> once on first container creation. Re-running against an existing database would produce duplicate-key errors on the <code>users.email</code> unique index. No production secrets in seed data — uses synthetic data (<code>admin@klearcom.local</code> , <code>+18005551234</code>).	<code>docker/mariadb/init.sql:49-78</code>

MongoDB (Document — `docker/mongodb/init.js` and `dev-api/src/mongo.js`)

Check	Finding	Evidence
Indexes	Present. <code>test_events</code> indexed on <code>{session_id, created_at}</code> ; <code>transcripts</code> indexed on <code>{module, reference_id, created_at}</code> . Created both in Docker init (<code>docker/mongodb/init.js:46-47</code>) and programmatically on connect (<code>dev-api/src/mongo.js:43-45</code>). <code>call_diagnostics</code> has an index in <code>dev-api/src/mongo.js:45</code> but not in the Docker init script — an inconsistency between the two environments.	<code>docker/mongodb/init.js:46-47</code> , <code>dev-api/src/mongo.js:43-45</code>
Schema validation	Absent. No JSON schema validation rules on any MongoDB collection. Documents are inserted with whatever shape the application sends — a mistyped field name or missing required field would be silently accepted.	No <code>db.createCollection(... validator)</code> calls anywhere in the repo
Seed data	Idempotent in dev-api — <code>seedMongoData()</code> checks <code>countDocuments</code> before inserting (<code>dev-api/src/mongo.js:84-88</code>) and supports a <code>--force</code> flag. Docker <code>init.js</code> runs once by design. No secrets in seed data.	<code>dev-api/src/mongo.js:84-95</code> , <code>docker/mongodb/init.js</code>

8.5 Development Environment

Check	Finding	Evidence
-------	---------	----------

<code>.env.example</code>	Partial. <code>backend/.env.example</code> (14 lines) and <code>dev-api/.env.example</code> (2 lines) are present and document required variables. However, there is no root <code>.env.example</code> and no <code>frontend/.env.example</code> — the frontend accepts <code>VITE_API_URL</code> via environment but the only documentation of this variable is in <code>docker-compose.yml:50</code> . A developer running the frontend standalone would not know this variable exists without reading Docker config or the <code>api/client.ts</code> source.	<code>backend/.env.example</code> present; <code>dev-api/.env.example</code> present; <code>frontend/.env.example</code> absent; root <code>.env.example</code> absent
OS portability	Good. Docker Compose provides a fully containerized path. The Quick Start section of <code>README.md</code> documents both Docker (<code>docker compose up --build</code>) and bare-metal (Node + PHP) paths. No OS-specific tooling is required. The dev-API uses Node 22 with <code>--env-file</code> flag (cross-platform). <code>entrypoint.sh</code> uses POSIX-compatible <code>#!/bin/sh</code> .	<code>README.md:24-48</code> , <code>docker-compose.yml</code> , <code>docker/php/entrypoint.sh:1</code>
Containerization	Present and current. <code>docker-compose.yml</code> defines 5 services: nginx (1.27-alpine), app (PHP 8.3-fpm with Composer 2), mariadb (11 with health check and retry), mongodb (7 with named volume), and frontend (Node 22-alpine). PHP Dockerfile installs <code>pdo_mysql</code> , <code>zip</code> , and <code>mongodb</code> extensions. Frontend Dockerfile runs the Vite dev server. MariaDB has a health check with <code>healthcheck.sh --connect --innodb_initialized</code> and 10 retries.	<code>docker-compose.yml:1-56</code> , <code>docker/php/Dockerfile:1-12</code> , <code>frontend/Dockerfile:1-6</code>
Code style enforcement	Absent. No ESLint, Prettier, PHP-CS-Fixer, Laravel Pint, or any formatter is configured anywhere. No <code>.editorconfig</code> . No pre-commit hooks (no <code>.husky/</code> , <code>.pre-commit-config.yaml</code> , <code>lint-staged</code> , or equivalent). PHPStan is declared in <code>backend/composer.json</code> but has no config file and is not run in CI. TypeScript <code>tsc --noEmit</code> catches type errors in CI but not style or quality issues. Mixed file extensions already exist (<code>LegacyMonitorPoller.jsx</code> alongside <code>.tsx</code> files) with no enforcement mechanism.	Searched entire repo for <code>.eslintrc*</code> , <code>eslint.config*</code> , <code>.prettier*</code> , <code>.editorconfig</code> , <code>phpcs.xml</code> , <code>pint.json</code> , <code>.php-cs-fixer*</code> , <code>phpstan.neon*</code> , <code>.husky/</code> , <code>lint-staged*</code> , <code>.pre-commit-config*</code> — all absent

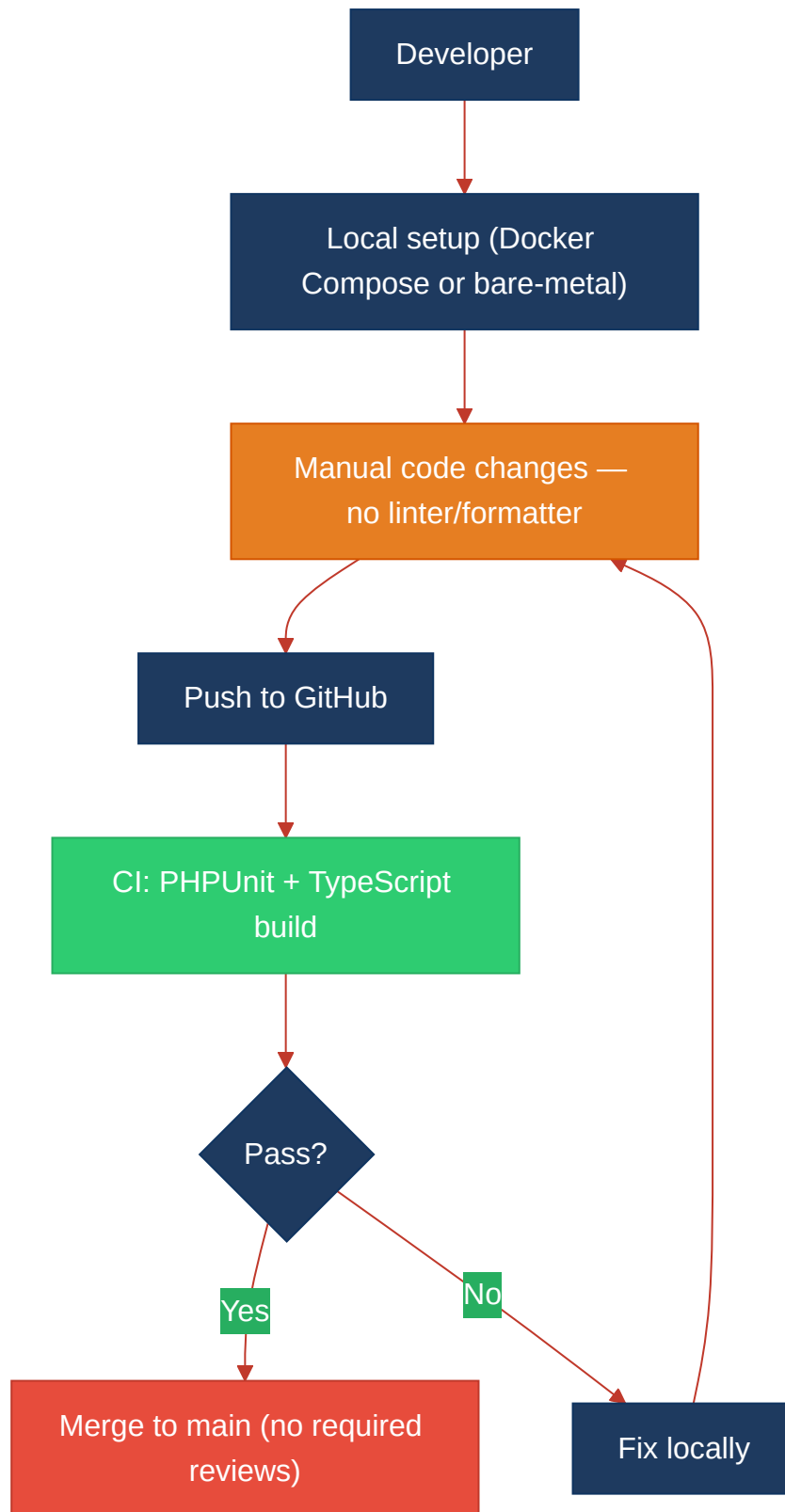
8.6 Prerequisites for Agentic Harness & Marketplace Readiness

Prerequisite	Current State	Gap
Enumerable work queue (clear, listable units of refactor work)	Two product modules (Discovery, Connect) with symmetric structure; <code>AGENTS.md</code> per module lists endpoints, stores, conventions. <code>docs/CODEBASE_AUDIT_ISSUES.md</code> enumerates 14 categorized issues with file paths and line numbers.	LOW — work queue exists in documentation; needs formalization into trackable items (GitHub Issues or Jira tickets) for agent consumption
Isolated, verifiable units of work	Each controller, service, and component is a single file with clear boundaries. Modules share no state beyond the database.	MEDIUM — units are isolated, but verification is weak: test suite is trivial, no snapshot/contract tests, no integration test harness
CI gate to accept agent-authored output	GitHub Actions CI runs PHPUnit and <code>tsc + vite build</code> on every PR.	HIGH — CI exists but provides low confidence: no linter gate, no static analysis (PHPStan unconfigured), tests are non-

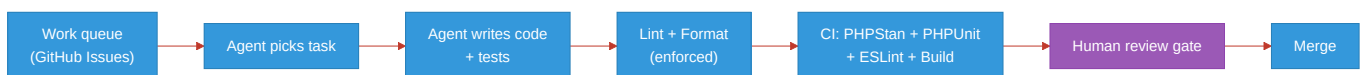
		deterministic or trivial, no <code>composer.lock</code> for reproducible builds
Repo hygiene for automation (clean checkout, no secrets)	<code>.gitignore</code> covers <code>.env</code> , vendor dirs, build output. No committed secrets found. Node lock files committed.	MEDIUM — mostly clean; commit <code>composer.lock</code> and add <code>frontend/.env.example</code> for full determinism
Marketplace packaging readiness	Docker Compose packages the full stack. Module boundaries are documented. No Helm chart, Terraform, or cloud-native deployment manifest.	MEDIUM — local packaging solid; cloud/marketplace packaging not started

8.7 Diagrams

Current dev / delivery flow



Agentic harness readiness target



Improvement roadmap



8.8 Actions Required

Gap	Action	Rating	Priority
<code>composer.lock</code> not committed	Run <code>composer install</code> locally to generate <code>composer.lock</code> , commit it. CI already runs <code>composer install</code> which will then use the lock file for reproducible builds.	MODERATE	CRITICAL
No linter or formatter configured	Add ESLint + Prettier for frontend/dev-api and PHP-CS-Fixer (or Laravel Pint) for backend. Add format-check steps to <code>.github/workflows/ci.yml</code> . Optionally add a pre-commit hook via Husky.	MODERATE	CRITICAL
PHPStan declared but unconfigured	Create <code>backend/phpstan.neon</code> at level 5+, add <code>vendor/bin/phpstan analyse</code> step to the CI workflow backend job. This gives the codebase static analysis coverage for free since the dependency is already installed.	MODERATE	HIGH
No branch protection artifacts	Add <code>.github/CODEOWNERS</code> mapping <code>backend/</code> and <code>frontend/</code> to respective owners, add <code>.github/PULL_REQUEST_TEMPLATE.md</code> , and configure required status checks + review on <code>main</code> via GitHub settings.	MODERATE	HIGH
No secondary indexes on MariaDB tables	Add indexes on <code>discovery_jobs(status)</code> , <code>discovery_nodes(discovery_job_id)</code> , <code>connect_monitors(status, country_code)</code> , <code>connect_check_results(connect_monitor_id, checked_at)</code> to avoid full table scans as data grows.	MODERATE	HIGH
No migration system — single <code>init.sql</code>	Introduce Laravel migrations or a numbered SQL migration runner. Split <code>init.sql</code> into per-table migration files with up/down. Add a CI step that applies migrations against a clean database.	MODERATE	HIGH
Missing <code>frontend/.env.example</code>	Create <code>frontend/.env.example</code> with <code>VITE_API_URL=http://localhost:8080/api</code> to document the required environment variable for standalone frontend development.	MODERATE	MEDIUM
MongoDB collections lack schema validation	Add <code>db.createCollection()</code> with JSON schema validator rules for <code>transcripts</code> , <code>test_events</code> , and <code>call_diagnostics</code> to prevent malformed documents from being silently accepted.	MODERATE	MEDIUM
<code>discovery_nodes.parent_id</code> has no FK constraint	Add <code>FOREIGN KEY (parent_id) REFERENCES discovery_nodes(id) ON DELETE CASCADE</code> to prevent orphan nodes in the IVR tree structure.	MODERATE	MEDIUM

<code>dotenv</code> declared but unused in <code>dev-api</code>	Remove <code>dotenv</code> from <code>dev-api/package.json</code> dependencies since Node 22's <code>--env-file</code> flag handles <code>.env</code> loading natively. Reduces dependency footprint.	MODERATE	LOW
---	---	----------	-----

8.9 Expected Outcomes

- **Reproducible builds across all workspaces** — committing `composer.lock` and adding missing `.env.example` files ensures any developer or CI runner can produce identical builds from a clean checkout.
- **Enforced code style as an automated quality gate** — ESLint, Prettier, and PHP-CS-Fixer in CI reject inconsistent code before review, enabling agent-authored PRs to be validated mechanically.
- **PHPStan + improved test suite provides a meaningful CI trust signal** — static analysis catches type errors and unreachable code; deterministic tests verify business logic, giving agents (and humans) confidence that passing CI means correct behavior.
- **Versioned migrations make schema changes safe and reversible** — a migration runner with up/down steps replaces the monolithic `init.sql`, enabling incremental schema evolution without manual intervention or full container recreation.
- **Foundation for agentic harness adoption** — with enforced style, reproducible builds, and a trusted CI gate, the codebase can safely accept machine-authored changes through an automated work-queue → agent → CI → human-review pipeline.